Day 66 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 4 reference

Watch: Simple Explanation of GRU (Gated Recurrent Units) | Deep Learning Tutorial 37 (codebasics) | Gated Recurrent Unit (GRU) Equations Explained (DataMListic)

`day66_gru_from_scratch.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graph are the ones the script saved. A glossary is at the end.

MATH BOX LEGEND

Symbols: z = update (keep) gate · r = reset gate · n = candidate · h = hidden state (the GRU's only state) · W's columns: [z | r | n], rows: [x | h]

Segment 0 — Setup

Imports and the reused pieces

CODE

```
import time
import numpy as np
from threadpoolctl import threadpool_limits
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, Circle
```

- `from threadpoolctl import threadpool_limits` — keeps NumPy's matrix library on one thread inside main(): same results, faster here.
- `sigmoid, softmax, clip_by_global_norm, make_sign_task` — copied verbatim from Days 57-65.
- `lstm_step_forward` — copied from Day 63; used only to time one LSTM step next to one GRU step in Section 6.

Segment 1 — The GRU cell

gru_step_forward and init_gru

MATH

$$z = sigmoid([x, h]\, W_z + b_z), \quad r = sigmoid([x, h]\, W_r + b_r)$$

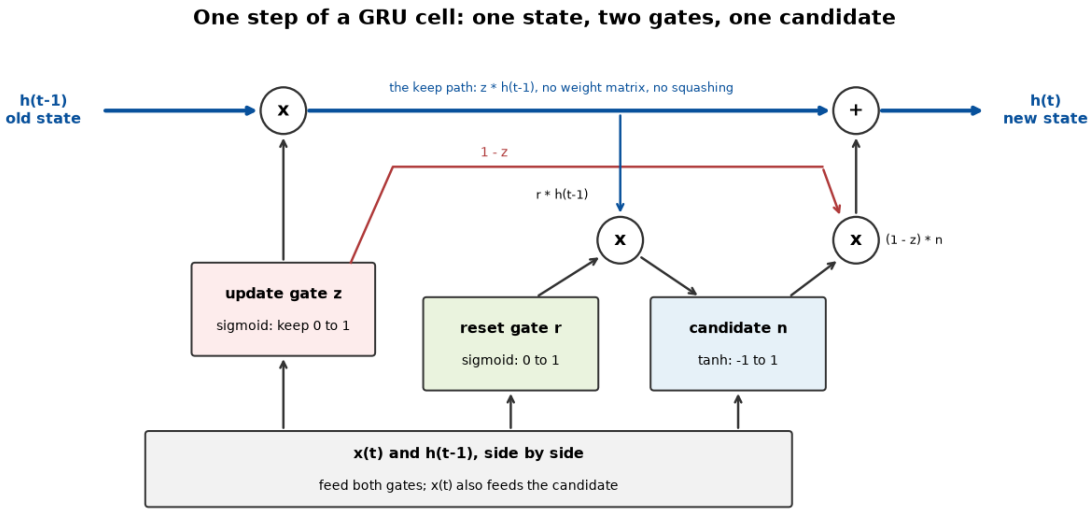
$$n = tanh(x\, W_{xn} + (r \cdot h)\, W_{hn} + b_n), \quad h_{new} = z \cdot h + (1 - z) \cdot n$$

CODE

```
def gru_step_forward(x_t, h_prev, W, b):
    D, H = x_t.shape[1], h_prev.shape[1]
    zr = np.concatenate([x_t, h_prev], axis=1) @ W[:, :2 * H] + b[:2 * H]
    z = sigmoid(zr[:, :H])
    r = sigmoid(zr[:, H:])
    n = np.tanh(x_t @ W[:D, 2 * H:] + (r * h_prev) @ W[D:, 2 * H:] + b[2 * H:])
    h = z * h_prev + (1.0 - z) * n
    return h, (z, r, n)
def init_gru(input_dim, hidden_dim, seed, keep_bias=0.0):
    rng = np.random.RandomState(seed)
    W = rng.randn(input_dim + hidden_dim, 3 * hidden_dim) / np.sqrt(input_dim + hidden_dim)
    b = np.zeros(3 * hidden_dim)
    b[:hidden_dim] = keep_bias
    return W, b
```

- `D, H = x_t.shape[1], h_prev.shape[1]` — input size and hidden size from the arrays' column counts.
- `W[:, :2 * H]` — the first 2H columns: the update gate's block and the reset gate's block.
- `zr[:, :H], zr[:, H:]` — split the two gates' raw scores.
- `W[:D, 2 * H:], W[D:, 2 * H:]` — the candidate's input weights and hidden weights, used separately because the hidden part multiplies r * h_prev.
- `(r * h_prev) @ W[D:, 2 * H:]` — the reset gate scales the old state entry by entry before it reaches the candidate.
- `z * h_prev + (1.0 - z) * n` — keep part of the old state, fill the rest with the candidate: one gate does both.
- `b[:hidden_dim] = keep_bias` — the update gate's biases: today's version of the LSTM's forget bias.

SYSTEM DIAGRAM



gru_step_forward as a picture: the keep path along the top, the reset gate feeding the candidate, and 1 - z deciding how much candidate is written.

Segment 2 — Forward and backward over sequences

gru_forward

CODE

```
def gru_forward(X, h0, W, b, mask=None):
    m, T, D = X.shape
    H = h0.shape[1]
    if mask is None:
        mask = np.ones((m, T))
    Hs = np.zeros((m, T, H))
    Z, R, N = (np.zeros((m, T, H)) for _ in range(3))
    h = h0
    for t in range(T):
        h_new, (z, r, n) = gru_step_forward(X[:, t], h, W, b)
        m_t = mask[:, t:t + 1]
        h = m_t * h_new + (1.0 - m_t) * h
        Hs[:, t] = h
        Z[:, t], R[:, t], N[:, t] = z, r, n
    cache = dict(X=X, h0=h0, W=W, b=b, mask=mask, H=Hs, Z=Z, R=R, N=N)
    return Hs, cache
```

- `Z, R, N = (np.zeros((m, T, H)) for _ in range(3))` — three separate arrays from a generator expression.
- `h_new, (z, r, n) = gru_step_forward(...)` — unpack the new state and the tuple of gates.
- `m_t * h_new + (1.0 - m_t) * h` — padding carries the state forward; the GRU has one state, so one line does it.
- `dict(X=X, h0=h0, ...)` — the cache for the backward pass.

gru_backward

MATH

$$dz = dh \cdot (h_{prev} - n), \quad dn = dh \cdot (1 - z), \quad dh_{prev} \leftarrow dh \cdot z$$

$$da_n = dn \cdot (1 - n^2); \quad d(r{\cdot}h) = da_n \, W_{hn}{}^{T}; \quad dr = d(r{\cdot}h) \cdot h_{prev}; \quad dh_{prev} \mathrel{+}= d(r{\cdot}h) \cdot r$$

$$[da_z, da_r] \text{ through the sigmoids; } [dx, dh_{prev}] \mathrel{+}= [da_z, da_r] \, W_{zr}{}^{T}$$

CODE

```
def gru_backward(dH, dh_last, cache):
    X, h0, W, mask = cache["X"], cache["h0"], cache["W"], cache["mask"]
    Hs, Z, R, N = cache["H"], cache["Z"], cache["R"], cache["N"]
    m, T, D = X.shape
    H = h0.shape[1]
    dW, db, dX = np.zeros_like(W), np.zeros(W.shape[1]), np.zeros_like(X)
    dh = dh_last.copy()
    for t in reversed(range(T)):
        dh = dh + dH[:, t]
        m_t = mask[:, t:t + 1]
        h_prev = Hs[:, t - 1] if t > 0 else h0
        z, r, n, x = Z[:, t], R[:, t], N[:, t], X[:, t]
        d = dh * m_t
        dz = d * (h_prev - n)
        dn = d * (1.0 - z)
        dh_prev = d * z + dh * (1.0 - m_t)
        da_n = dn * (1.0 - n ** 2)
        rh = r * h_prev
        dW[:D, 2 * H:] += x.T @ da_n
        dW[D:, 2 * H:] += rh.T @ da_n
        db[2 * H:] += da_n.sum(axis=0)
        d_rh = da_n @ W[D:, 2 * H:].T
        dr = d_rh * h_prev
        dh_prev += d_rh * r
        da_zr = np.concatenate([dz * z * (1.0 - z), dr * r * (1.0 - r)], axis=1)
        zin = np.concatenate([x, h_prev], axis=1)
        dW[:, :2 * H] += zin.T @ da_zr
        db[:2 * H] += da_zr.sum(axis=0)
        dzin = da_zr @ W[:, :2 * H].T
        dX[:, t] = dzin[:, :D] + da_n @ W[:D, 2 * H:].T
        dh = dh_prev + dzin[:, D:]
    return dW, db, dX, dh
```

- `d = dh * m_t` — only real steps pass gradient into their own gates.
- `dz = d * (h_prev - n)` — h = z*h_prev + (1-z)*n, so changing z moves h by (h_prev - n).
- `dh_prev = d * z + dh * (1.0 - m_t)` — the direct route: times the update gate; padding passes dh straight through.
- `dW[:D, 2 * H:] += x.T @ da_n` — accumulate into one block of W, picked by slicing rows and columns.
- `d_rh = da_n @ W[D:, 2 * H:].T` — gradient for the product r * h_prev ...
- `dr = d_rh * h_prev; dh_prev += d_rh * r` — ... split by the product rule between r and h_prev.
- `np.concatenate([dz * z * (1.0 - z), dr * r * (1.0 - r)], axis=1)` — both gates' gradients through their sigmoids, side by side in z, r order.
- `dX[:, t] = dzin[:, :D] + da_n @ W[:D, 2 * H:].T` — the input fed both the gates and the candidate, so its gradient has two parts.
- `dh = dh_prev + dzin[:, D:]` — all routes back to h_prev added together.

GRUClassifier

CODE

```
class GRUClassifier:
    def __init__(self, input_dim, hidden_dim, n_classes, seed=7, keep_bias=1.0, lr=0.05,
        clip=1.0):
        self.W, self.b = init_gru(input_dim, hidden_dim, seed, keep_bias)
        rng = np.random.RandomState(seed + 1)
        self.Why = rng.randn(hidden_dim, n_classes) / np.sqrt(hidden_dim)
        self.by = np.zeros(n_classes)
        self.H, self.lr, self.clip = hidden_dim, lr, clip
    def forward(self, X, mask=None):
        self.Hs, self.cache = gru_forward(X, np.zeros((X.shape[0], self.H)), self.W, self.b, mask)
        self.h_last = self.Hs[:, -1]
        self.probs = softmax(self.h_last @ self.Why + self.by)
        return self.probs
    def loss(self, y):
        return -np.mean(np.log(self.probs[np.arange(len(y)), y] + 1e-12))
    def gradients(self, y):
        m = len(y)
        dlogits = self.probs.copy()
        dlogits[np.arange(m), y] -= 1.0
        dlogits /= m
        grads = dict(Why=self.h_last.T @ dlogits, by=dlogits.sum(axis=0))
        grads["W"], grads["b"], self.dX, _ = gru_backward(np.zeros_like(self.Hs), dlogits @
            self.Why.T, self.cache)
        return grads
    def train_step(self, X, y):
        self.forward(X)
        grads, _, _ = clip_by_global_norm(self.gradients(y), self.clip)
        for name, g in grads.items():
            getattr(self, name)[...] -= self.lr * g
    def accuracy(self, X, y):
        return (self.forward(X).argmax(axis=1) == y).mean()
```

- `Day 65's LSTMClassifier with the cell swapped` — same output layer, loss, clipping and update rule, so training results can be compared directly.
- `self.Hs[:, -1]` — the state after each sequence's last real step (padding carries it).
- `grads["W"], grads["b"], self.dX, _ = gru_backward(...)` — four return values; the gradient at h0 is not needed.
- `getattr(self, name)[...] -= self.lr * g` — update each weight array in place, chosen by its name.

Segment 3 — Checks

numeric_grad and report (from Day 65)

CODE

```
def numeric_grad(f, P, eps=1e-5):
    numeric = np.zeros_like(P)
    for pos in np.ndindex(P.shape):
        old = P[pos]
        P[pos] = old + eps; up = f()
        P[pos] = old - eps; down = f()
        P[pos] = old
        numeric[pos] = (up - down) / (2 * eps)
    return numeric
def report(name, numeric, analytic):
    diff = np.abs(numeric - analytic)
    rel = diff / (np.abs(numeric) + np.abs(analytic) + 1e-12)
    ok = bool(np.all(diff <= 1e-10 + 1e-6 * np.abs(numeric)))
    print(f"{name:>7}{numeric.size:>9}{rel.max():>20.2e}{diff.max():>18.2e}{str(ok):>8}")
    return ok
```

- `verbatim from Day 65` — the same checker and the same pass rule as the LSTM's checks.

- `np.ndindex(P.shape)` — every index of an array of any shape.
- `bool(np.all(...))` — one True/False for the whole array.

Segment 4 — Training

train_runs

CODE

```
def train_runs(T, kb, lr, seeds=(7, 8, 9), epochs=150):
    X_tr, y_tr = make_sign_task(600, T, seed=42)
    X_te, y_te = make_sign_task(400, T, seed=999)
    X_tr, X_te = X_tr[:, :, None], X_te[:, :, None]
    runs = []
    for seed in seeds:
        net = GRUClassifier(1, 16, 2, seed=seed, keep_bias=kb, lr=lr, clip=1.0)
        rng = np.random.RandomState(seed)
        curve = []
        for epoch in range(epochs):
            order = rng.permutation(600)
            for start in range(0, 600, 30):
                idx = order[start:start + 30]
                net.train_step(X_tr[idx], y_tr[idx])
            if epoch % 10 == 0 or epoch == epochs - 1:
                curve.append(net.accuracy(X_te, y_te))
        runs.append((max(curve), curve[-1]))
    return runs
```

- `seeds=(7, 8, 9), epochs=150` — default arguments; the follow-up section changes epochs for one setting.
- `X_tr[:, :, None]` — (600, T) becomes (600, T, 1): one input per step.
- `if epoch % 10 == 0 or epoch == epochs - 1:` — test accuracy every 10 epochs and at the end, whatever the budget.
- `runs.append((max(curve), curve[-1]))` — peak and final for one seed.

The grid and the count of runs that learned

CODE

```
learned = {kb: sum(f >= 0.9 for (T, k, lr), runs in results.items() if k == kb for _, f in runs)
    for kb in [0.0, 1.0, 2.0]}
```

- `{kb: sum(... for ... in ... if ... for ...) for kb in [...]}` — a dictionary comprehension around a generator with two loops: for each keep-bias, count every final accuracy of at least 0.9 across all lengths and learning rates.
- `results.items()` — pairs of (key, runs); the key (T, k, lr) is unpacked in the loop.

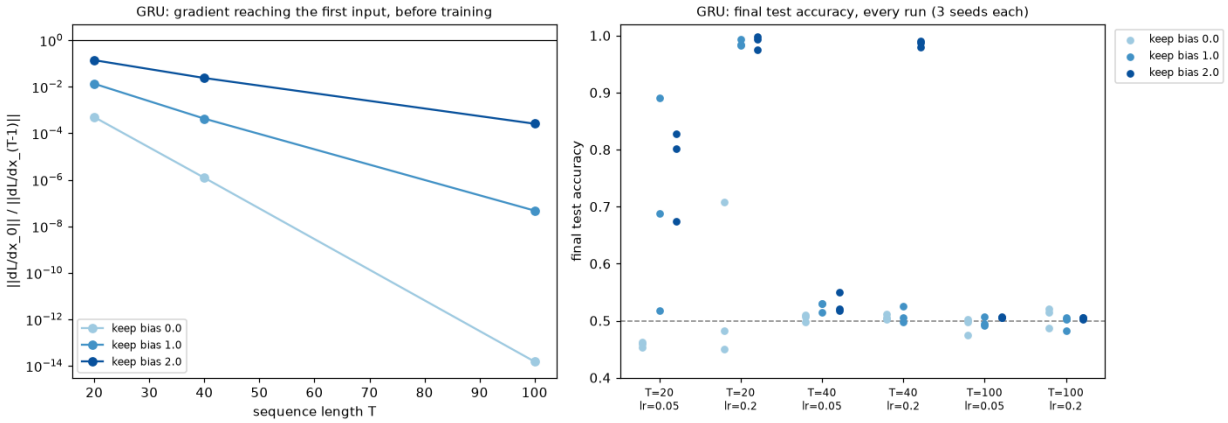
The follow-up at 40 steps

CODE

```
for label, T, kb, lr, epochs in [("T=40, keep-bias 2, lr 0.05, 450 epochs", 40, 2.0, 0.05, 450),
                                 ("T=40, keep-bias 3, lr 0.2", 40, 3.0, 0.2, 150),
                                 ("T=40, keep-bias 4, lr 0.2", 40, 4.0, 0.2, 150),
                                 ("T=100, keep-bias 3, lr 0.2", 100, 3.0, 0.2, 150),
                                 ("T=100, keep-bias 4, lr 0.2", 100, 4.0, 0.2, 150)]:
    runs = train_runs(T, kb, lr, epochs=epochs)
    rows[label] = runs
    cells = "  ".join(f"{p:.3f}/{f:.3f}" for p, f in runs)
    print(f"{label:<44}{cells:<40}{sum(f >= 0.9 for _, f in runs):>9} of 3")
```

- `for label, T, kb, lr, epochs in [...]:` — each item is a 5-tuple: a description and the four settings of that run.
- `train_runs(T, kb, lr, epochs=epochs)` — the same training function, with the settings of this row.
- `{label:<44}` — left-aligned text in 44 characters, so the columns line up.

REAL GRAPH



Real graph from the script: the GRU's gradient reach before training, and the final accuracy of every run on Day 65's grid.

Segment 5 — demo_cost

Timing one step of each cell

CODE

```
def best(fn):
    times = []
    for _ in range(5):
        start = time.perf_counter()
        for _ in range(20000):
            fn()
        times.append(time.perf_counter() - start)
    return min(times)
t_gru = best(lambda: gru_step_forward(x, h, Wg, bg))
t_lstm = best(lambda: lstm_step_forward(x, h, c, Wl, bl))
```

- `def best(fn):` — a helper defined inside the demo; fn is a function with no arguments.
- `lambda: gru_step_forward(x, h, Wg, bg)` — wraps the call so best() can repeat it without knowing its arguments.
- `min(times)` — the fastest of five timings is the least disturbed by other work.

Quick Syntax Glossary — new or reinforced constructs

- `W[:D, 2 * H:]` — slice rows and columns at once to pick one block of a stacked weight matrix.
- `z * h + (1 - z) * n` — an interpolation: z decides how much of each side to take.
- `dW[block] += ...` — accumulate a gradient into one block of a larger array.
- `def f(..., epochs=150):` — a default argument a caller can override by name.
- `lambda: f(a, b)` — a zero-argument function that calls f with fixed arguments.
- `{k: sum(... for ...) for k in ...}` — count things per key in one expression.
- `(np.zeros(s) for _ in range(3))` — several separate arrays, unpacked into several names.

End of Day 66 syntax reference. For the experiments and what they showed, see the Day 66 complete guide.